Отчёт по программе «Канатная дорога»

Исполнитель: Крылосов Сергей Андреевич

Группа: БПИ 253

Вариант: 46

Условие задачи

Между двумя станциями непрерывно движутся кабины канатной дороги. Пассажиры прибывают на обе станции и ждут кабину нужного направления. Вместимость каждой кабины ограничена. Посадка и высадка выполняются только тогда, когда кабина находится на станции. После прибытия сначала выходят пассажиры, достигшие станции назначения, затем садятся ожидающие. Оператор разрешает отправление после закрытия дверей и завершения операций. При сильном ветре или технической неисправности движение останавливается; пассажиры остаются учтёнными до возобновления работы или эвакуации.

Требуется консольное приложение, которое моделирует работу дороги. Входные параметры включают число и вместимость кабин, время пути, количество пассажиров и частоту их появления, время посадки и высадки, категории приоритета, вероятность и длительность остановок, погодные интервалы и продолжительность работы. В журнале должны отображаться прибытие пассажиров и кабин, высадка, посадка, отказ из-за заполнения, закрытие дверей, отправление, движение, остановка, возобновление работы и доставка пассажиров. Вместимость кабин не должна превышаться; посадка и высадка возможны только на станции; кабина не отправляется с открытыми дверями; остановка не меняет состав кабин сама по себе.

По условию работа завершается после закрытия входа и доставки принятых пассажиров либо после аварийной эвакуации. SIGINT/SIGTERM запрашивают штатную эвакуацию и завершение потоков.

Объекты предыдущего проекта и их реализация потоками

Все основные объекты предметной области взяты из предыдущего проекта HA1: станции, кабины, пассажиры и оператор. В HA2 их поведение реализовано потоками: по отдельному pthread на станцию, кабину и принятого пассажира; роль оператора выполняет главный поток main. Вспомогательные структуры Settings, Statistics, очереди и журнал остаются данными, а не отдельными потоками.

Объект из HA1	Реализация в HA2	Действия потока
Станция	station_worker, два потока	Принимает пассажиров в очередь, выбирает VIP раньше обычных пассажиров, допускает прибывшие и резервные кабины, освобождает места, эвакуирует очередь
Кабина	cabin_worker, один поток на кабину	Хранит реальный состав кабины, отсчитывает время операций, начинает посадку и высадку, сообщает об окончании пути, закрывает двери и запрашивает разрешение отправления
Пассажир	passenger_worker, один поток на принятого пассажира	Выбирает станцию и категорию билета, сам регистрируется на станции, проверяет и подтверждает переходы ожидание → посадка → поездка → доставка либо эвакуация

Объект из НА1	Реализация в НА2	Действия потока
Оператор	главный поток main и service_step	Управляет модельным временем, приёмом пассажиров, остановками, разрешением отправления и завершением работы

Взаимодействие потоков

Станция хранит VIP-очередь, обычную очередь, очередь кабин и очередь ожидающих допуска. На станции три места обслуживания. Прибывшие кабины получают место раньше резервных. Кабина владеет своим состоянием, таймером time_left и списком пассажиров. Дублирующие массивы очередей и состава кабин из прежних потоков удалены.

У каждого участника есть FIFO-очередь запросов inbox и FIFO-очередь ответов replies. Связные списки защищены pthread_mutex_t, а семафоры будят получателя. Потоки получают запросы через mailbox, выполняют собственные правила и возвращают результат. Например, кабина запрашивает у станции пассажира; станция выбирает его по приоритету и возвращает номер и категорию. Пассажир проверяет своё состояние и подтверждает начало посадки. Завершив посадку, кабина закрывает двери и возвращает оператору запрос отправления. Оператор проверяет отсутствие остановки и разрешает рейс отдельным сообщением.

Один шаг service_step соответствует одной модельной секунде. Оператор последовательно передаёт участникам право выполнить действие и ждёт ответ перед следующим зависимым действием. Внутренние обращения идут от кабины к станции или пассажиру, от нового пассажира к станции; станции не ждут ответов от кабин. При эвакуации станция обращается к уже зарегистрированным пассажирам, которые не посылают новых запросов станции. Циклических ожиданий в этом протоколе нет.

Память модели общая, но в каждый момент изменять её может только текущий исполнитель запроса. Отправитель ждёт ответ и не читает изменяемые данные до его получения. Такое упорядочение через mutex и semaphore защищает также журнал, статистику и генератор случайных чисел. Это простая имитация с распределением поведения по потокам, а не попытка ускорить вычисления параллельной записью в Service. Одновременно неподтверждённый запрос к одному участнику может быть только один; внешний код не должен обходить этот протокол.

При сильном ветре или поломке таймеры операций не уменьшаются, состав кабин сохраняется, а модельное время продолжает идти. Входящие пассажиры могут ждать на станциях. При завершении оператор получает ответы об эвакуации или доставке, закрывает очереди, присоединяет все созданные потоки через pthread_join и освобождает память.

Максимум создаётся 214 потоков участников: две станции, 12 кабин и 200 пассажиров; вместе с главным потоком — 215. Потоки доставленных пассажиров ждут закрытия очереди до завершения модели.

Поток создаётся вызовом pthread_create(&actor->thread, NULL, worker, actor). worker — функция потока, actor — указатель на его структуру с типом участника, номером и очередями запросов и ответов. runtime_init() запускает потоки двух станций и заданного числа кабин. runtime_start_passenger() создаёт поток пассажира при его появлении. Если pthread_create() завершается ошибкой, программа сообщает об ошибке и останавливает уже созданные потоки через pthread_join().

Пользовательский интерфейс

Программа запускается в терминале. Сначала она последовательно запрашивает настройки: число кабин и их вместимость, общий лимит пассажиров, продолжительность приёма с 09:00, время пути, параметры поломок, частоту и размер групп прибывающих пассажиров, время посадки и высадки, вероятность VIP-билета и число интервалов сильного ветра. Для каждого интервала вводятся начало и конец в секундах от 09:00; начало включено в интервал, конец не включён. Допускается от 0 до 3 интервалов ветра. Нулевое число означает отсутствие погодных остановок.

На каждый вопрос вводится одно значение и нажимается Enter. Числа проверяются на допустимый диапазон; при неверном ответе выводится ошибка и повторяется тот же вопрос. Дробные вероятности вводятся через точку, например 0.2. Вопросы выводятся в стандартный поток ошибок (stderr), ответы читаются из стандартного ввода (stdin), а журнал событий выводится в стандартный поток вывода (stdout) с помощью файловых дескрипторов и вызовов read()/write(). Поэтому ./cable_car > logs.txt сохраняет события в файл, оставляя вопросы в терминале.

Журнал содержит модельное время в формате [чч:мм:сс], описание операций и итоговые счётчики принятых, доставленных и эвакуированных пассажиров. Между группами событий есть пауза в одну реальную секунду; она нужна для чтения журнала и не равна шагу модельного времени.

Тестирование и исправление ошибок

Для проверки модели используются тесты из tests/service_test.c, которые запускаются командой make test. Они проверяют точное время посадки, поездки и высадки, приоритет VIP, вместимость станции в три кабины, остановку из-за ветра, завершение уже начатого рейса, вызов резервных кабин и эвакуацию. Дополнительно выполняются 30 прогонов с разными исходными значениями генератора случайных чисел. На каждом шаге проверяется, что кабины находятся в допустимых состояниях, число мест не превышено и каждый принятый пассажир учтён ровно один раз — в очереди, при посадке, в кабине, среди доставленных или эвакуированных. Реальная задержка вывода в тестах отключена, чтобы она не замедляла проверку модельного времени.

Примеры проверяемых ситуаций (время t отсчитывается в секундах от 09:00):

Сценарий	Исходные данные	Ожидаемый и полученный результат
Станция заполнена	Четыре кабины с одним пассажиром каждая одновременно заканчивают путь к A; высадка одного пассажира занимает 10 секунд	Три кабины занимают места и начинают высадку, четвёртая ждёт перед станцией. При t = 11 доставлены три пассажира, четвёртая кабина допущена на станцию; при t = 21 доставлен четвёртый
Ветер во время посадки	Одна кабина, посадка длится 10 секунд; ветер действует на интервале [5, 15)	Таймер посадки не идёт во время остановки: кабина отправляется при t = 20, заканчивает путь при t = 50, пассажир доставлен при t = 60
Закрытие приёма во время рейса	Приём закрывается при t = 1, пустая кабина уже находится в пути, до конца пути осталось 30 секунд	Программа не завершается при закрытии приёма и ждёт окончания рейса; завершение происходит при t = 30

Запуск make test завершается сообщением Service tests passed. В тестах специально проверяются и ошибочные настройки; сообщения SOFT_ASSERT для них ожидаемы.

При просмотре первых журналов обнаружилась задержка высадки: кабина уже допускалась на станцию, двери открывались, но пассажиры оставались в ней ещё несколько минут. Причина заключалась в порядке обслуживания нескольких кабин. Теперь после допуска каждой кабины высадка начинается сразу, и у каждой кабины отдельно отсчитывается время операции. Тест с четырьмя одновременно прибывающими полными кабинами проверяет оба случая: три занимают места и начинают высадку одновременно, четвёртая ждёт перед заполненной станцией с закрытыми дверями и начинает высадку после освобождения места. Такое ожидание перед станцией допускается; ожидание с открытыми дверями после допуска на станцию — нет.

Исправлены и другие ошибки, замеченные по журналам: посадка больше не завершается мгновенно и не получает лишнюю секунду задержки; при завершении рабочего дня программа дожидается прибытия кабин, уже находящихся в пути; резервная кабина вызывается только тогда, когда на её станции после распределения пассажиров осталась очередь. Проверки этих ситуаций закреплены в тестах, чтобы ошибки не появились снова. Тесты с заведомо неверными настройками специально вызывают сообщения SOFT_ASSERT; такие сообщения ожидаемы и сами по себе не означают провал тестов.

Запуск и завершение

В папке проекта программа собирается командой make и запускается командой ./cable_car или make run. Команда make test запускает проверки модели. Чтобы сохранить журнал, можно выполнить ./cable_car > logs.txt. Значения настроек можно подготовить в текстовом файле по одному на строку и запустить ./cable_car < settings.txt > logs.txt.

Поведение зависит от введённых данных:

Исходные данные или ситуация	Результат
Лимит пассажиров равен 0	Новые пассажиры не появляются; после окончания времени приёма кабины выводятся на стоянку и работа завершается
Пассажиры приняты, остановок нет	Они проходят очереди, посадку и поездку; после закрытия входа уже принятые пассажиры доставляются, начатые рейсы заканчиваются
Указан интервал сильного ветра	На это время операции приостанавливаются и затем продолжаются с сохранённого состояния
Случилась техническая неисправность	Движение и обслуживание приостанавливаются на заданную длительность; при совпадении с ветром работа возобновляется только после устранения обеих причин
После закрытия входа обслуживание не завершилось за 12 модельных часов	Выполняется автоматическая эвакуация оставшихся пассажиров, затем моделирование завершается
Ввод закончился раньше времени или произошла ошибка ввода-вывода	Программа завершается с ошибкой до запуска модели

Обычное завершение наступает после закрытия приёма, учёта всех принятых пассажиров как доставленных и вывода всех кабин на стоянку. При SIGINT или SIGTERM диспетчер инициирует эвакуацию оставшихся пассажиров. При штатном или аварийном завершении каждому actor-потоку отправляется сигнал закрытия, затем вызывается pthread_join(); память mailbox и actor-массивов освобождается.

Структура многопоточной версии

• src/components/service.c — оператор, модельное время, остановки и координация запросов

• src/components/station_worker.c — реальные очереди станции, выбор пассажиров и допуск кабин

• src/components/cabin_worker.c — операции кабины, таймер, пассажиры и запрос отправления

• src/components/passenger_worker.c — появление пассажира, регистрация и проверка переходов

• src/threads/runtime.c — запуск потоков, передача запросов, получение ответов и завершение

• src/threads/mailbox.c — FIFO-очередь сообщений на mutex и semaphore

• include/threads — структуры участников, запросов, ответов и очередей

• src/inputs, src/queues, include/components — ввод, журнал и структуры предыдущего проекта

Проверка многопоточной реализации

После рефакторинга выполнена команда make all test с UndefinedBehaviorSanitizer и предупреждениями как ошибками. Сохранены 30 пошаговых прогонов прежней модели. Новые tests/worker_test.c используют настоящие pthread, mutex и semaphore: проверяют ответы станции с VIP-приоритетом, отклонение неправильного перехода пассажира, 12 завершённых прогонов с 1–12 кабинами, остановку из-за ветра и эвакуацию на 11 разных шагах. Получены сообщения Service tests passed и Worker tests passed.

Дополнительно эти тесты прошли с ThreadSanitizer без сообщений о гонках данных. Проверены SIGINT и SIGTERM у собранного консольного приложения: оба сигнала приводят к эвакуации и успешному завершению. Журналы тестов находятся в build/tests/service.log и build/tests/worker.log.

Требования ТЗ и ограничения реализации

Проект предназначен для Linux и использует C, pthread, mutex, semaphore и неименованный семафор POSIX через sem_init, sem_wait и sem_post. Вызов sem_init создаёт семафор с начальным счётчиком 0 для потоков одного процесса.

Замечание о потоках, которые только отражают уведомления, устранено: предметные решения выполняются обработчиками участников, есть обратные ответы и взаимодействие между потоками. Вычисления зависимых шагов намеренно упорядочены для простоты и воспроизводимости. Погодные пороги пока заменены интервалами ветра; пустые кабины ждут спроса вместо непрерывного движения. Эти отдельные отличия от буквального сценария в данном рефакторинге не менялись.

Использование ИИ и критическая оценка

ИИ использовался при проектировании архитектуры, реализации actor runtime, mailbox, синхронизации завершения и документации. Промты, относящиеся к этой работе:

> Основываясь на функционале из ДЗ AKOS_HA_1, собери проект, разбивая его на логические модули.
>
> Сверь архитектуру с файлом ТЗ по многопоточности в новой папке.
>
> Реализуй полный код в соответствии с архитектурой и ТЗ.

Критическая оценка: прежняя версия создавала потоки, но оставляла предметные решения в service_step. При повторной проверке это признано недостатком. Правила перенесены в

обработчики станций, кабин и пассажиров; уведомления заменены запросами с ответами, удалены копии состояния, добавлены проверки реальных потоков и ThreadSanitizer. Оператор сохраняет порядок модельного времени, но больше не выбирает пассажира из очереди и не выполняет операции кабины. Ограничение — последовательная обработка зависимых действий: многопоточность моделирует роли, а не ускоряет расчёт.

Оригинальные промты по подготовке README исходной версии сохранены ниже для контекста:

> Ты — опытный C-разработчик и технический писатель. На основе предоставленного проекта на языке C подготовь подробную документацию в формате README.md на русском языке.
>
> Документация должна быть структурирована и написана в стиле технического задания/руководства пользователя. Не выдумывай поведение программы: сначала проанализируй исходный код, структуры данных, функции, тесты, Makefile и конфигурацию проекта.
>
> напиши README.md на русском языке для проекта «Канатная дорога» на C. Это консольная дискретно-событийная модель между станциями A и B,начало работы в 09:00. Обязательно опиши сборку make, запуск make run / ./cable_car, очистку make clean, ввод 12 параметров и всю логику работы модели.Параметры: количество кабин 1–12, вместимость кабины 1–20, максимальное количество пасажиров 0–200, длительность рабоыт 1–43200 секунд , время пути 1–1800 секунд, длительность поломки 1–300 секунд, интервал появления пассажиров 1–3600 секунд, количество пассажиров за интервал 1–20, время посадки одного пассажира 1–60 секунд, время высадки одного пассажира 1–60 секунд, вероятнсть VIP 0–1, вероятность поломки за минуту 0–0.5. целочисленные параметры должны быть положительными кроме общего лимита пассажиров, которому разрешён 0 дробные значения вводятся через точку. Опиши что 0 отключает соотвествующее событие,VIP=1 означает что все пассажиры VIP. Первая группа появляется сразу в 09:00 следующие группы появляются через заданный интервал например при интервале 60 секунд и 5 пассажирах группы — в 09:00, 09:01 ,09:02 и т.д., при этом последняя группа ограничивается оставшимся лимитом . Станция и VIP-статус пассажира выбираются слуайно. На кажой станции максимум 3 места для кабин остальные кабины находятся в резерве.на каждой станции есть очередь кабин и две очереди пассажиров - VIP и обычная. VIP имеют приоритет при посадке но занимают обычные места. При допуске кабины сначала выполняется высадка, затем посадка VIP и обычных пассажиров пассажир закрепляется за кабиной в начале посдки а прибывший позже VIP не может занять место уже начавшейся операци. Внутри каждой очереди сохраняется порядок прибытя. Все места станции обслуживаются параллельно , вместимость кабин и станций не превышается. после посадки кабина отправляется на другую станцию если там нет места она ждёт перед станцией с закрытыми дверями.Отдельно различай окончание времени пути и фактический допуск на станцию. Прибывшие кабины получают освободившиеся места раньше резервных . Пустые кабины не совершают бессмысленных рейсов и после закрытия входа направляются только за ожидающими пасажирами после чего выводятся на стоянку. Посадка и высадка не должны получать скрытую дополнительную секунду: если один пассажир обслуживается 5 секунд операция из 5 пассажиров занимает 25 секунд. Поломка проверяется раз в модельную минуту то есть каждые 60 модельных секунд ; при поломке таймеры кабин останавливаются, модельное время продолжает идти а после заданной длительности поломки движение возобновляется.При вероятности поломки 0 событие отключено. По умолчанию ветра нет но интервалы ветра могут задаваться через Settings.wind; во время опасного интервала движение преостанавливается. Журнал группирует события с одинаковым модельным временем например [09:00:00] , и между группами в обычном запуске есть задержка 1 реальная секунда. Объясни что модельная секунда не равна реальной секунде. В журнале должны быть прибытие пассажиров VIP/обычный статус посадка высадка, отправление окончание пути допуск на станцию заполнение кабин, остановки и возобновление движения. Для сохранения журнала укажи ./cable_car > log.txt и ./cable_car < settings.txt > log.txt. Когда заданная длительность работы от 09:00 заканчивается приём новых пассажиров закрывается но уже принятые пассажиры продолжают обслуживание и начатые рейсы завершаются. после закрытия входа все кабины выводятся на стоянку когда пассажиров больше нет. Предусмотри максимальное время завершения 12 модельных часов после закрытия входа ; при превышении оставшиеся пассажиры эвакуируются через service_evacuate(). В конце выведи количество принятых доставленных и эвакуированных пасажиров. Опиши структуру проектн app/main.c, src/inputs, src/queues, src/components, include, tests, использование read()/write() , освобождение памяти и тесты make test / make test-c. Укажи что тесты проверяют закрепление пассажира в начале посадки точную длительность опреаций параллельную высадку, максимум 3 кабин на станции, ожидание перед заполненной станцией пустые рейсы, остановки и эвакуацию а также выполняют 30 прогонов с разным колчеством кабин. Объясни что сообщения SOFT_ASSERT при намеренно неправильных тестовых параметрах ожидаемы и

успешное завершение обозначается Service tests passed.Не выдумывай функции и поведение котрых нет в проектн. ФАЙЛЫ ПРОЕКТА Для БОЛЕЕ КАЧЕСТВЕННОГО РЕДМИ ПРИЛАГАЮ НИЖЕ

Дополнительный запрос на исправление архитектуры:

исправь этот пункт **Самостоятельность потоков требует доработки или согласования.** service_step принимает все решения и меняет основное состояние. Потоки сущностей лишь обрабатывают уведомления и поддерживают копии состояния; ответов диспетчеру нет. Формальные потоки и асинхронные очереди присутствуют, но самостоятельное поведение участников реализовано слабо. Для устранения этого замечания нужен отдельный архитектурный рефакторинг, а не усложнение текущих очередей. + добавь в отчет что все объекты взят с предыдущего проекта реализованы потоки